Exercise #PT-M1: Multinomial Classification Using Logistic Regression and Decision Trees

Leonardo Antero SP. Yoro
IT Elective III, BSIT
Jose Rizal University
Mandaluyong, Philippines
leonardoantero.yoro@my.jru.edu

***Abstract*— This study tackles the effect of hyperparameter tuning on the performance of Logistic Regression and Decision Tree models, specifically with respect to multinomial classification tasks. Adjusting such hyperparameters as regularization strength, solver type, tree depth, and data-splitting criteria impacts a model's accuracy, precision, recall, and F1 score. The results showed that tuning of hyperparameters enhances Decision Tree performance with some instances of unstable increases in variance compared to Logistic Regression performance, which is much more stable. This finding points out the necessity of strategic tuning of hyperparameters toward improving modeling performance and generalization.**

Keywords*— *Multinomial classification, Logistic Regression, Decision Tree, Hyperparameter Tuning, Machine learning

I. Introduction

Logistic Regression and Decision Tree classifiers are machine learning's indispensable tools for predictive modeling. Logistic Regression is employed in binary classification, whereas Decision Trees offer a versatile method for classification and regression modeling with complex relationships in the data. Logistic Regression is a supervised learning model that makes a prediction of the probability of a categorical dependent variable based on independent variables, one or more in number [2]. It applies the logistic function to transform estimated values into a probability ranging from 0 to 1 under the assumption that there is a linear relationship between the independent variables and the log-odds of the dependent variable. It is especially applicable to binary cases, such as yes or no cases. (GeeksforGeeks, 2025).

Decision Trees is a supervised learning algorithm that may be used in both classification and regression tasks [3]. They build a model that will predict the value of a target variable given more than one input feature, constructing a tree-based structure where every node is a decision made from feature values. Decision Trees can handle both numerical and categorical data with minimal preprocessing, but they tend to overfit, particularly with deep trees, and need methods such as pruning to enhance generalization (GeeksforGeeks, 2025).

Logistic Regression is optimal to apply in situations where the relationship is evidently linear and outcomes are binary, while Decision Trees are more general and can learn more complex relationships and thus can be applied to more general problems. Both models are evaluated using metrics such as accuracy, precision, recall, and F1-score for measuring performance in classification tasks. Studying these algorithms allows practitioners to choose the right model depending on the type of data they have and the requirements of their analysis.

II. Data Collection

1. *Dataset Description*

The research employs a weather classification dataset [1] with 13,200 instances and 11 attributes that are obtained from Kaggle, a popular data-sharing platform across multiple industries. The data set is appropriate for multinomial classification since it comprises a variety of numerical and categorical values that describe different weather conditions.

```
[ ] print("Initial shape of the dataset:", df.shape)

Initial shape of the dataset: (13200, 11)
```

Fig. 1. Initial Shape of the Dataset

```
[ ] print("Intial columns of the dataset:\n", df.columns)

Intial columns of the dataset:
 Index(['Temperature', 'Humidity', 'Wind Speed', 'Precipitation (%)',
       'Cloud Cover', 'Atmospheric Pressure', 'UV Index', 'Season',
       'Visibility (km)', 'Location', 'Weather Type'],
      dtype='object')
```

Fig. 2. Initial Columns of the Dataset

df.info() was also executed, in order to display the number of records, column names, its data types, memory usage, and non-null counts. The command helps the researchers to identify missing values and analyze the dataset structure. Df.dtypes() was also executed to display the data types of each attributes in the DataFrame. This further helps in data preprocessing and analysis as it determines whether the attribute contains numerical or categorical data.

```
[ ] df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 8 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Temperature            13200 non-null  float64
 1   Humidity               13200 non-null  int64
 2   Wind Speed             13200 non-null  float64
 3   Precipitation (%)      13200 non-null  float64
 4   Atmospheric Pressure   13200 non-null  float64
 5   UV Index               13200 non-null  int64
 6   Visibility (km)        13200 non-null  float64
 7   Weather Type           13200 non-null  object
dtypes: float64(5), int64(2), object(1)
memory usage: 825.1+ KB
```

Fig. 3. df.info() command execution

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

2. *Dataset Attributes*

The database has 13,200 instances with 11 attributes, and there are a mix of numerical and categorical features. Here are explanations of each attribute and its relationship to the study:

a) Temperature: Numerical attribute representing the temperature. Temperature is significant in weather categorization as it decides different weather conditions.

b) Humidity: Quantitative attribute representing air humidity percentage. Higher humidity is typically linked to overcast or rain.

c) Wind Speed: Wind patterns result in weather alterations and are of great importance when distinguishing between weather types.

d) Precipitation (%): Numerical feature indicating the possibility of precipitation in percentage. The greater the value, the higher the probability to rain or snow, and therefore it is a significant feature in classification.

e) Cloud Cover: Categorical attribute denoting the category of cloud cover (e.g., clear, partly cloudy, overcast). Cloud cover has immediate visual indicators to weather and aids in discriminating between sunny, cloudy, or rainy weather.

f) Atmospheric Pressure: Numerical attribute representing the air pressure. Pressure changes are highly related with weather condition changes.

g) UV Index: Quantitative feature that represents the intensity of ultraviolet radiation. The greater the UV index, the more commonly it occurs on sunny days and serves to differentiate between clear and cloudy days.

h) Season: Categorical feature for the season when weather was observed (e.g., Winter, Spring). Seasons are associated with numerous weather conditions and impact classification results.

i) Visibility (km): Numerical attribute for the range of visibility in kilometers. Low visibility is generally correlated with fog, rain, or snow and thus turns out to be helpful for classification.

j) Location: Categorical attribute specifying the location (e.g., coastal, inland, mountain). Various locations have their respective weather conditions that affect classification.

k) Weather Type: Categorical feature describing the type of weather condition being recorded (e.g., Sunny, Cloudy, Rainy). The feature shows what kind of weather is common at the time of data collection.

3. *Data Cleaning and Data Preprocessing*

The data cleaning and data preprocessing was conducted in Google Collaboratory, inside an .ipynb Jupyter Notebook file that was provided to the researcher.

a) Data preprocessing for further data cleaning

The 'Cloud Cover', 'Location', and 'Season' attribute was removed as it does not bring relevance to the objective of the study. It was then removed using this command:

```
[ ] df.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Atmospheric Pressure	UV Index	Visibility (km)	Weather Type
0	14.0	73	9.5	82.0	1010.82	2	3.5	Rainy
1	39.0	96	8.5	71.0	1011.43	7	10.0	Cloudy
2	30.0	64	7.0	16.0	1018.72	5	5.5	Sunny
3	38.0	83	1.5	82.0	1026.25	7	1.0	Sunny
4	27.0	74	17.0	66.0	990.67	1	2.5	Rainy

Fig. 4. Removal of the following attributes and the display of records after removal.

"Cloud Cover" is redundant because "Weather Type" already provides the same description, which can lead to multicollinearity.

"Season" is too general and indirectly correlated with day-to-day weather, as characteristics such as temperature and humidity already provide better coverage of seasonal variations

"Location" does not add useful numerical information and can result in overfitting, where the model learns regional weather by heart rather than general trends.

The command *df.hist(bins=50, figsize=(20,15))* generates histograms for all numerical attributes in the DataFrame. Each histogram plots the distribution of values in a column so that one may easily find the trends, skewness, or outliers. Here, *bins=50* sets the number of bins for grouping the values, *figsize=(20,15)* scales the figure for better readability. Finally, *plt.show()* draws the plots.

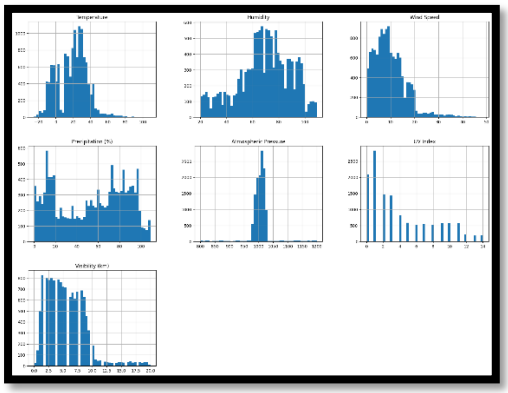


Fig. 5. Histogram display of the current, uncleaned attribtues

The following loop goes over each column of the DataFrame, providing deep insights in the dataset about the number of unique values and missing values within each attribute. The unique value count will inform us if the column is categorical or numerical data and evaluate how much variability is present in a numerical feature. Meanwhile, it will also highlight missing values and allow us to understand possible quality issues with our data and which handling technique will be suitable such as imputation or removal.

```
[ ] for column in df:
        print(f"Unique values in {column}:", df[column].nunique())
        print(f"Missing values in {column}:", df[column].isnull().sum())
        print()

Unique values in Temperature: 126
Missing values in Temperature: 0

Unique values in Humidity: 90
Missing values in Humidity: 0

Unique values in Wind Speed: 97
Missing values in Wind Speed: 0

Unique values in Precipitation (%): 110
Missing values in Precipitation (%): 0

Unique values in Atmospheric Pressure: 5456
Missing values in Atmospheric Pressure: 0

Unique values in UV Index: 15
Missing values in UV Index: 0

Unique values in Visibility (km): 41
Missing values in Visibility (km): 0

Unique values in Weather Type: 4
Missing values in Weather Type: 0
```

Fig. 6. Output of code for finding unique and missing values for each attribute

b) Identifying Missing Values

According to GeeksforGeeks (2024) [4], Managing missing values is important in machine learning because it keeps the data integrity and completeness, which affects the performance and accuracy of models. Practitioners can increase the reliability of analyses and quality of predictive models by properly dealing with missing data.

This code calculates the number of missing values in each column of the dataset and prints the results as a dictionary. It helps to identify data gaps that may need imputation or removal before further analysis.

```
[ ] missing_values = df.isnull().sum()
    print(missing_values.to_dict())

{'Temperature': 0, 'Humidity': 0, 'Wind Speed': 0, 'Precipitation (%)': 0, 'Atmospheric Pressure': 0, '
```

Fig. 7. Output of code for identifying missing values for each attribute

```
[ ] df.dropna(axis=1, how='all', inplace=True)

    print("Remaining columns:")
    print(df.columns)

Remaining columns:
Index(['Temperature', 'Humidity', 'Wind Speed', 'Precipitation (%)',
       'Atmospheric Pressure', 'UV Index', 'Visibility (km)', 'Weather Type'],
      dtype='object')
```

Fig. 8. Shape of the DataFrame after handling missing values

It has concluded that there were no missing values, upon executing various commands that helps the study get close to optimal data analysis, thus, moving forward to handling duplicate values and records. Finally, the figure above (Fig. 8) displays the current shape of the DataFrame after handling the missing values.

c) Identifying Duplicate Values

Future Processing (2023) [5] explains that it is crucial to detect and remove duplicate records from a dataset. Duplication of data adds to the risk of carrying out inconclusive studies. Quality of the dataset for modelling and analysis is improved by removing duplicates, ensuring each data point is distinct, and avoiding data redundancy. In the current dataset used and industry it was from, duplicate values may appear because of weather patterns becoming too similar and hence produce similar statistics.

The following code prints the number of unique values in each column within the dataset and includes the count of duplicates for each column, which gives insight into variability within the data and may suggest redundancy that should be cleaned and preprocessed.

```
[ ] for column in df.columns:
        print(f"Unique values in {column}:", df[column].nunique())
        print(f"Number of duplicates in '{column}' column: {df[column].duplicated().sum()} \n")

Unique values in Temperature: 126
Number of duplicates in 'Temperature' column: 13074

Unique values in Humidity: 90
Number of duplicates in 'Humidity' column: 13110

Unique values in Wind Speed: 97
Number of duplicates in 'Wind Speed' column: 13103

Unique values in Precipitation (%): 110
Number of duplicates in 'Precipitation (%)' column: 13090

Unique values in Atmospheric Pressure: 5456
Number of duplicates in 'Atmospheric Pressure' column: 7744

Unique values in UV Index: 15
Number of duplicates in 'UV Index' column: 13185

Unique values in Visibility (km): 41
Number of duplicates in 'Visibility (km)' column: 13159

Unique values in Weather Type: 4
Number of duplicates in 'Weather Type' column: 13196
```

Fig. 9. Lines of code used for identifying unique and duplicate values for each attribute

The next code shows which columns do not have duplicated values in their rows, thus all the columns contain unique values within each column. This happens if the column's unique count of values equals the total row count in the data set. The execution displays that there are no columns or attributes where all its records are unique, signifying existing duplicate values.

```
unique_columns = []
for column in df.columns:
    if df[column].nunique() == len(df):
        unique_columns.append(column)

print("Columns where each row is unique:")
print(unique_columns)
```

```
Columns where each row is unique:
[]
```

Fig. 10. Output of code for identifying columns where each row is unique

The researchers then identified all unique values in each attribute. It helps understand the distribution of categorical variables, detect anomalies, and identify potential data inconsistencies. This is very useful for feature selection and data cleaning because it allows you to check for unexpected values or patterns in the dataset.

```
for column in df.columns:
    print(f"Unique values in column '{column}':")
    print(df[column].unique())
    print()
```

```
Unique values in column 'Temperature':
[ 14.  39.  30.  38.  27.  32.  -2.   3.  28.  35.  12. -10.  24.  10.
  33.  43.  13.  -7.  26.   4.  17.  40.   2.  15.  29.  11.  -9.  36.
  42.  21.  22.  25.  -4.  -1.  -5.  41.  31.  16.  34.  49.  19.  23.
  20.  -3.  18.   1.   0.  46.  44. -13.  -6.  78.  63.  73.   8. -12.
 -24.  -8.  60.  48.   5.  51. -14.  50.  37.  54.  47.  70.   9.  66.
 -16. -15.  59.  80. -19.  52.  45.   6. -18. -11.  74.  76.  55. -20.
  57.  91.  82. -17.  61.   7.  53.  65.  77.  67.  64.  58.  68.  72.
  62.  71.  56. 107. -22.  75.  85.  97.  84. -21.  92. -25.  81. 109.
  98.  94.  90. -23.  88.  99.  69. 100.  89. 102.  86. 108.  87.  95.]

Unique values in column 'Humidity':
[ 73  96  64  83  74  55  97  85  45  43  59  87  21  50  27  51  46 102
  67  88  36  79  72  57  61  70  95  69  90 105  49  37  22  54  66  25
  91  98  94  41  84  63  75  52  89  47  81  62  31  68  35  78  56  93
  44  38  24  82  65  80  39  48  60  29  99  92  76  77  86  32  58  42
  30 100  33  71 107 108  26 106  28 109 101  34 103  40  23 104  53  20]

Unique values in column 'Wind Speed':
[ 9.5  8.5  7.   1.5 17.   3.5  8.   6.   2.  10.5 15.   6.5  0.5 12.
 12.5  7.5 13.5  1.   4.  16.  16.5  2.5 23.   3.  10.  25.5 19.  11.5
  0.   9.  18.5 11.  20.  14.   5.5 13.  46.5  5.  18.  28.5 14.5  4.5
 15.5 28.  19.5 21.5 34.  17.5 47.  34.5 35.5 23.5 42.5 33.  31.5 26.
 22.  36.5 27.5 20.5 35.  30.  26.5 21.  32.5 32.  24.  27.  22.5 31.
 30.5 24.5 29.5 37.  44.5 41.  41.5 40.5 37.5 46.  25.  39.  29.  45.
 43.5 45.5 36.  38.  44.  38.5 33.5 40.  42.  47.5 39.5 43.  48.5]

Unique values in column 'Precipitation (%)':
[ 82.  71.  16.  66.  26.  86.  96. 107.  25.  67.   8.  46.  13.  27.
  15.  72.  75.  98.  18.  29. 101.  85.  62.  12.   3.  54.  97.  63.
  56.  91.   6.  76. 109.  58.  37.   9.   0.  39.  11.  14.  88.  78.
  55.  90.   4.  69.  68.  32.  87.  17.  89.  57.  41.  84.  83.  47.
  99.  92.  19.  60.  65.   5.  43.   2.  59.  79.  94.  42.  53.  73.
  81.  52.  36.  45.  21.  22.  40.  50.  51.  10.  35.  95. 103.   7.
  74.  48.  61.  64.  31. 106.  93.  24.   1.  80.  23.  70.  20. 104.
  77.  30.  33.  38.  34.  49.  28. 100.  44. 108. 105. 102.]

Unique values in column 'Atmospheric Pressure':
[1010.82 1011.43 1018.72 ... 1022.86 1067.23  984.27]

Unique values in column 'UV Index':
[ 2  7  5  1  0  8 11  3  9  4 13 10 14  6 12]

Unique values in column 'Visibility (km)':
[ 3.5 10.   5.5  1.   2.5  5.   4.   7.5  1.5  8.5  6.   8.   3.   9.5
  9.   4.5  2.  16.5 12.5  6.5  7.   0.  17.5 17.  13.  11.   0.5 16.
 18.  10.5 11.5 19.  18.5 13.5 15.5 15.  14.5 14.  12.  20.  19.5]

Unique values in column 'Weather Type':
['Rainy' 'Cloudy' 'Sunny' 'Snowy']
```

Fig. 11. Code execution for displaying all unique values in each attribute

This line verifies how many duplicate rows are in the dataset. As it printed "0", that means there are no identical rows. I concluded that I will not be removing any duplicate values because, although some of the individual columns contain repeated values, they are not row duplicates, so they still contain valuable information for analysis.

In weather records, repeated values of the same temperature, humidity, or wind speed occur on various days and times. No entire row is being repeated, so each record continues to contribute uniquely to the data. It is possible that eliminating these values may cause loss of valuable variations in weather, which are critical to making an accurate classification model.

```
df.duplicated().sum()
```

```
0
```

Fig. 12. Code execution of checking of existing duplicate rows

d) Outlier Detection

Future Processing (2023) [5] also states that outliers are data points that differ significantly from most values and can distort analysis results if not handled properly. Identifying outliers is done using statistical methods or visualization tools. Once identified, their causes should be investigated to determine if they are genuine or data entry errors. Treatment options depend on analysis goals, such as removing, transforming, or treating them as missing values. Addressing outliers ensures more accurate and reliable model performance.

The researchers first used a code called *detect_outliers_iqr(df)* for detecting numerical outliers for each attribute inside the dataset, which is then visualized through boxplots. The code generated boxplots for each attribute, efficiently detecting outliers such as the boxplot for Temperature in Fig. 13, helping the researcher on what to do next with these outliers.

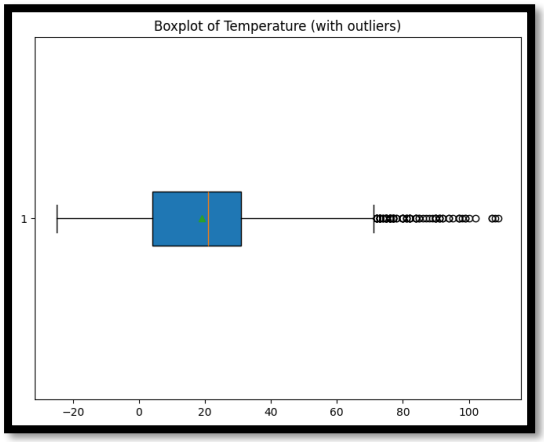


Fig. 13. Outlier detection with sample boxplot

The cleaning then moves forward by removing outliers with the *remove_outliers_iqr(df)* function, filtering out the extreme values, and then visualized again through boxplots, in order to use a more normalized distribution of data.

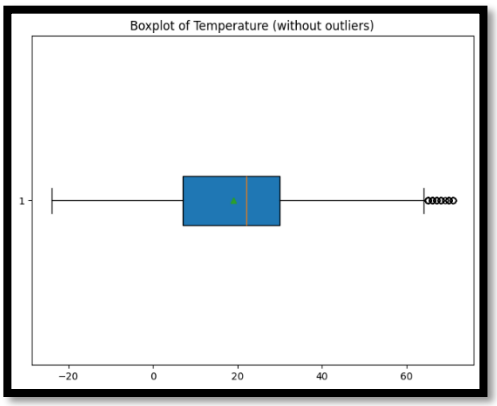


Fig. 14. Visualization of the same boxplots after outlier removal

After the removal of outliers, the shape of the dataset is checked again, now, as its cleaned version that will be used for attribute selection and model building, although, there are remaining outliers. Some outliers are retained to preserve the data's integrity and to ensure that the model captures all potential variations in weather patterns.

```
[ ] df = df_cleaned.copy()  # Keep the actual cleaned dataset

    print("Shape of the cleaned dataset::", df.shape)

    Shape of the cleaned dataset:: (11689, 8)

[ ] print("Columns of the cleaned dataset::\n", df_cleaned.columns)

    num_columns = len(df_cleaned.columns)

    print("\nNumber of columns in the dataset after cleaning:", num_columns)

    Columns of the cleaned dataset::
     Index(['Temperature', 'Humidity', 'Wind Speed', 'Precipitation (%)',
            'Atmospheric Pressure', 'UV Index', 'Visibility (km)', 'Weather Type'],
           dtype='object')

    Number of columns in the dataset after cleaning: 8
```

Fig. 15. The current shape, existing columns, and number of columns in the dataset after data cleaning

Finally, the shape and remaining columns of the dataset was checked, with 11689 records and 8 columns worth of processed data, ready to be used for further experimentation.

e) Dataset training and testing sets

The data was divided into training and test sets based on an 80-20 ratio for Multinomial Logistic Regression (MLR) models. k-fold cross-validation was also used to enhance the stability of performance metrics. For MLR, k-fold cross-validation was employed to minimize variance and enhance generalizability by validating and training on various subsets. For DT, k-fold cross-validation was utilized to reduce bias and variance for consistent evaluation on all data points.

Training data is utilized to instruct the machine learning model to recognize patterns and make judgments, while testing data measures the extent to which the model can generalize to novel, unseen data. The bigger the training dataset, the better the model is likely to be, as more patterns and variations are exposed to the model [6]. K-Fold Cross-Validation reinforces this further by dividing the data into more subsets to be used for training and validation, minimizing bias and variance. This allows each point of data to be utilized for both training and testing, giving a more accurate and balanced estimate of performance. By measuring performance over multiple folds, the model stabilizes and is less sensitive to particular data splits, resulting in better generalization and accuracy [7].

III. Model Building: Logistic Regression vs. Decision Tree Classifier

Hyperparameter tuning allows for significantly better performance as these parameters control the learning and training phase of the algorithm. According to IBM [9], hyperparameters are parameters that are set before the training of the model. They affect the function of the learning algorithm and significantly influence the model generalization capability. Tuning of parameters properly avoids the overfitting of a model where the model picks up important patterns without being too complex. By fine tuning the hyperparameters, the analysts achieve a good balance between variance and bias leading to better predictions.

Tuning hyperparameters lets data scientists try out various model designs and settings to locate the optimal mixes for their data and tasks. This boosts the model's prediction strength, alongside improving its reliability and usefulness.

On the other hand, Data visualization plays an important role in machine learning as it can help simplify complex data, making it easier to understand and draw conclusions based on. Also, better communication occurs when data is represented in a way that it gives more context to it, even to those who are not data scientists. When trends and patterns can be identified more easily, decisions can be made quicker.

Visualizations also provide better understanding of data because they display all the different relationships among the variables, even those that do not stand out when the data is not processed yet, and this helps with Exploratory Data Analysis (EDA) and modeling development. The overall approach of visualizing a methodology makes the analysis process simplified and leads to a better informed, data-driven result.

a) Multinomial Logistic Regression (MLR)

One of the most effective methods of studying relationships between independent variables and one or more dependent categorical variables that take on three or more levels of a variable in their measurement is multinomial logistic regression. This method is used most

commonly in marketing, healthcare, and social sciences to analyze group differences and to predict various types of outcomes. Researchers can learn from it about the probability that different outcomes are likely by predicting each of the categories' probabilities, which were based on independent variables. In addition, recent studies by Una'a-Hermosilla et. al. (2020) [8] present clear proofs of real-life application in providing rich results of analyses on complex categorical data.

The Multinomial Logistic Regression (MLR) model was built using the LogisticRegression class of sklearn.linear_model, which can be utilized for multi-class classification tasks. The target variable "Weather Type" was encoded using LabelEncoder to convert categorical labels into numerical values to make it compatible with the model.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold, cross_val_score, train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

# Load dataset
file_path = "cleaned - weather_classification_data.csv"
df = pd.read_csv(file_path)

# Features and target variable
X = df.drop(columns=["Weather Type"])
y = df["Weather Type"]

# Encode target variable
le = LabelEncoder()
y = le.fit_transform(y)
```

Fig. 16. Code preparation and target variable initialization

Data was split into an 80-20 ratio by using train_test_split. The numerical features of the training set were scaled through StandardScaler in order to increase the model's convergence ability through scaling input values.

```
# Split data for evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Fig. 17. Testing data and training data splitting and StandardScaler function

The baseline model was initialized with the following hyperparameters:

C = 1.0 – Controls the strength of regularization; lower values increase regularization and reduce overfitting.

solver = "lbfgs" – The solver is well-suited to large datasets.

max_iter = 1000 – Specifies the upper bound on iterations allowed for convergence by the solver to reduce premature stoppages.

```
# Define multinomial logistic regression model
model = LogisticRegression(C=1.0, solver="lbfgs", max_iter=1000) #multi_class="multinomial"
```

Fig. 18. MLR baseline model definition

To estimate the performance, 5-fold cross-validation was applied, where the model was trained on four of the folds and tested on one. This was done five times, and the average accuracy on all the folds was calculated to get a better estimation of the model's performance. The value of k is adjusted in other experimentation to experiment with its influence on the accuracy of the model.

```
# Implement k-fold cross-validation (k=5)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train_scaled, y_train, cv=kf, scoring="accuracy")
```

Fig. 19. K-Fold Cross-Validation function

After cross-validation, the model was then fitted on the whole training set using model.fit() and tested on the test set using model.predict(). A confusion matrix was used to show predictive accuracy by classes. A classification report showed the precision, recall, and F1-score, giving a detailed breakdown of the model's performance.

```
model.fit(X_train_scaled, y_train)

y_pred1 = model.predict(X_test_scaled)
```

Fig. 20. Training and testing steps

A confusion matrix makes it easier to understand how well a model is performing by illustrating correct and incorrect predictions. It also facilitates calculation of important metrics such as accuracy, precision, and recall, which provide a better indication of performance, particularly when dealing with imbalanced data (Geeksforgeeks, 2025) [9].

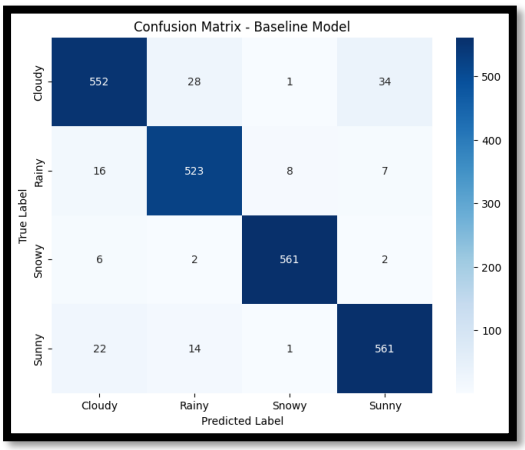


Fig. 21. Confusion matrix utilization

```
Classification Report - Baseline Model:
              precision    recall  f1-score   support

           0       0.93      0.90      0.91       615
           1       0.92      0.94      0.93       554
           2       0.98      0.98      0.98       571
           3       0.93      0.94      0.93       598

    accuracy                           0.94      2338
   macro avg       0.94      0.94      0.94      2338
weighted avg       0.94      0.94      0.94      2338
```

Fig. 22. Classification Report Utilization

Finally, cross-validation accuracies were plotted through a line graph to display model consistency across folds and ensure that the model performed well and generalized perfectly across splits in data.

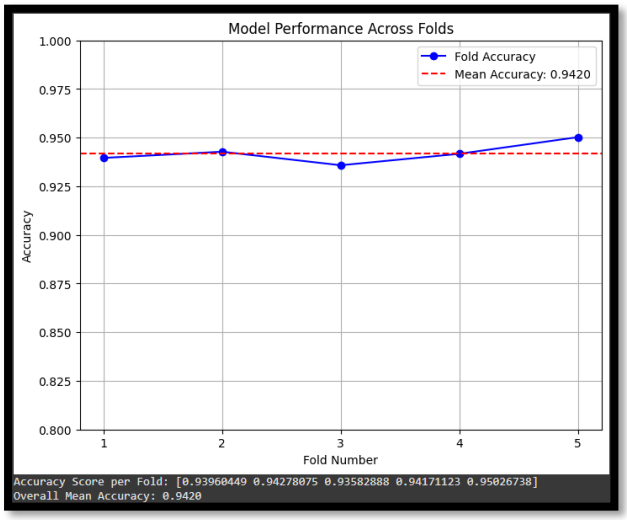


Fig. 23. Baseline Performance Across Folds

With the MLR model prepared, the researcher then conducted experimentation through hyperparameter tuning using the following:

- C value
- solver
- max_iter
- k-fold value (n_splits)

Among multiple experiments done, the following models are the baseline MLR model and the best MLR model logged in the experimentation phase.

I.) Baseline MLR Model

The baseline MLR model contains the following parameters, with the visualization of the results being Fig. 21 to Fig. 23:

- C = 1
- solver = "lbfgs"
- max_iter = 1000
- n_splits = 5

The model was assessed on 5-fold cross-validation and attained an average accuracy of 94.20% with similar performance for each fold, indicative of good generalization. High true positive rates were noted using the confusion matrix but recognized misclassification trends for Cloudy vs. Sunny with 34 and 22 errors, respectively, and Rainy to Cloudy with 16 errors. The classification report indicates a balanced precision, recall, and F1-scores with 94%, and Snowy had the highest recall at 98%. Overall, the model had good classification strength, but differentiation between similar weather conditions is a point of improvement.

II.) Best MLR model with weak regularization

The alternative MLR model, experiment 8, contains the following parameters:

- C = 100
- solver = "lbfgs"
- max_iter = 100
- n_splits = 5

The 8th multinomial logistic regression model conducted with a C value of 100 and a lbfgs solver had an overall accuracy of 94.24% using 5-fold cross-validation. The accuracy scores across the folds were similar, from 93.53% to 95.13%, which reflects consistent performance across subsets.

The confusion matrix identifies that the model can distinguish among the weather conditions correctly. Misclassifications, however, include between Cloudy and Sunny with 34 instances of Cloudy being mislabeled as Sunny and 22 instances of Sunny being mislabeled as Cloudy. Snowy class had maximum precision and recall with very little misclassifications.

The classification report shows robust and balanced performance among all classes with accuracy, recall, and F1-scores at approximately 94%. The model shows regular predictive strength along the categories, without significant class imbalance or the overfitting problem.

The 8th model preserves the highest accuracy and stability, dependable for the classification of weather conditions, without significant classification uncertainty. Although, it is using a weak regularization value.

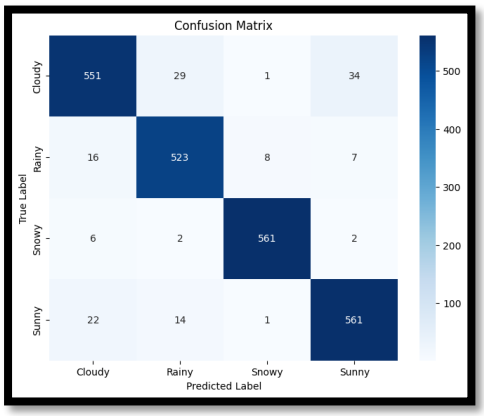


Fig. 24. Confusion matrix of the MLR model 8

```
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.90      0.91       615
           1       0.92      0.94      0.93       554
           2       0.98      0.98      0.98       571
           3       0.93      0.94      0.93       598

    accuracy                           0.94      2338
   macro avg       0.94      0.94      0.94      2338
weighted avg       0.94      0.94      0.94      2338
```

Fig. 25. Classification report of the MLR model 8

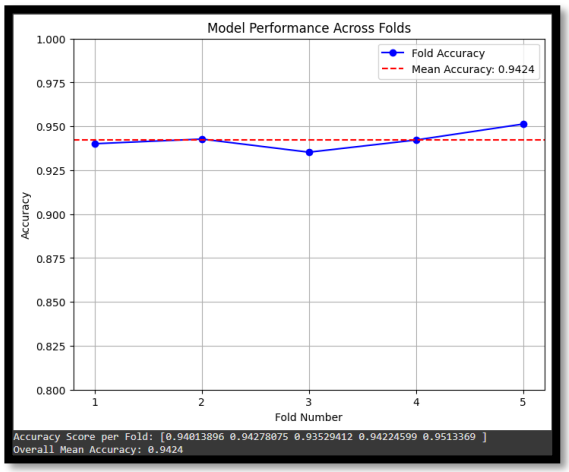


Fig. 26. Model performance across folds of the MLR model 8 and its overall mean accuracy

III.) Best MLR model with strong regularization

The 6th model, experiment 6, contains the following parameters:

- C = 1
- solver = “saga”
- max_iter = 5000
- n_splits = 5

With a total accuracy of 94% and a mean cross-validation accuracy of 0.9420, Experiment 6 shows consistent and reliable performance on different validation folds with accuracies between 0.9358 and 0.9523. This consistency shows that the model can generalize well to new data and lower the chance of overfitting. The classification report indicates high and balanced precision, recall, and F1-scores for all classes with a score of 0.94, which indicates the stable performance of the model in precisely predicting all types of weather.

The confusion matrix confirms the same, indicating lower misclassifications, as most predictions were correctly positioned on the diagonal. This shows that the model can differentiate between weather types with low errors. Using 1 for C's value during regularization in Experiment 6 finds the perfect equilibrium between complexity and performance without causing overfitting but while being accurate enough. The pair of the "saga" solver and the maximum iteration as 5000 helps ensure enough convergence, even in the case of bigger data, and hence goes towards providing strong performance of the model, as low maximum iteration values often struggle to converge.

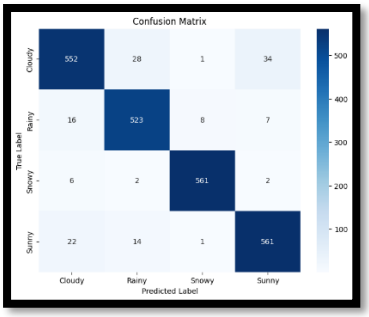


Fig. 27. Confusion matrix of the MLR model 6

```
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.90      0.91       615
           1       0.92      0.94      0.93       554
           2       0.98      0.98      0.98       571
           3       0.93      0.94      0.93       598

    accuracy                           0.94      2338
   macro avg       0.94      0.94      0.94      2338
weighted avg       0.94      0.94      0.94      2338
```

Fig. 28. Classification report of the MLR model 6

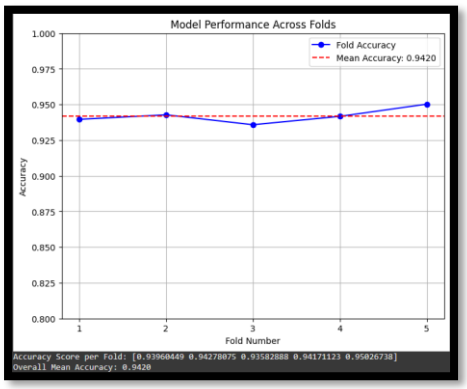
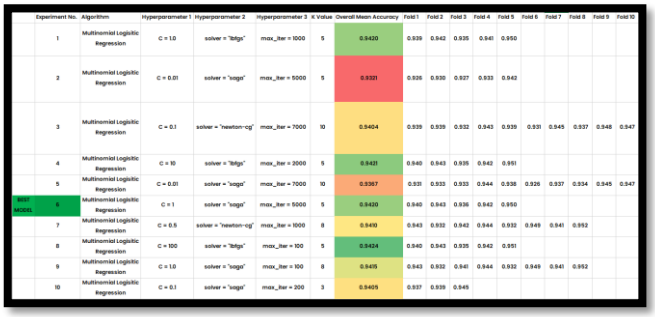


Fig. 29. Model performance across folds of the MLR model 6 and its overall mean accuracy

IV.) Experiments Assessment

The following figure then shows the complete experiment logs, utilizing conditional formatting to visualize the best and worst accuracies and containing each fold accuracy:



	Experiment No.	Algorithm	Hyperparameter 1	Hyperparameter 2	Hyperparameter 3	K Value	Overall Mean Accuracy	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Fold 6	Fold 7	Fold 8	Fold 9	Fold 10
	1	Multinomial Logistic Regression	C = 1.0	solver = "lbfgs"	max_iter = 1000	5	0.9420	0.939	0.942	0.935	0.941	0.950					
	2	Multinomial Logistic Regression	C = 0.01	solver = "saga"	max_iter = 5000	5	0.9331	0.926	0.930	0.927	0.933	0.942					
	3	Multinomial Logistic Regression	C = 0.1	solver = "newton-cg"	max_iter = 7000	10	0.9404	0.939	0.939	0.932	0.943	0.939	0.931	0.945	0.937	0.948	0.947
	4	Multinomial Logistic Regression	C = 10	solver = "lbfgs"	max_iter = 2000	5	0.9421	0.940	0.943	0.935	0.942	0.951					
	5	Multinomial Logistic Regression	C = 0.01	solver = "saga"	max_iter = 7000	10	0.9367	0.931	0.933	0.933	0.944	0.938	0.926	0.937	0.934	0.945	0.947
BEST MODEL	6	Multinomial Logistic Regression	C = 1	solver = "saga"	max_iter = 5000	5	0.9420	0.940	0.943	0.936	0.942	0.950					
	7	Multinomial Logistic Regression	C = 0.5	solver = "newton-cg"	max_iter = 1000	8	0.9410	0.943	0.932	0.942	0.944	0.932	0.949	0.941	0.952		
	8	Multinomial Logistic Regression	C = 100	solver = "lbfgs"	max_iter = 100	5	0.9424	0.940	0.943	0.935	0.942	0.951					
	9	Multinomial Logistic Regression	C = 1.0	solver = "saga"	max_iter = 100	8	0.9415	0.943	0.932	0.941	0.944	0.932	0.949	0.941	0.952		
	10	Multinomial Logistic Regression	C = 0.1	solver = "saga"	max_iter = 200	3	0.9405	0.937	0.939	0.945							

Fig. 30. Multinomial Logistic Regression Experiment Logs

Out of the 10 experiments, Experiment 6 is the best option as it was found to have the best balance between accuracy, efficiency, and stability of validations. Although Experiment 1 had a robust baseline, Experiment 6 not only replicated the same 94.20% accuracy but utilized "saga", which is more efficient when dealing with big datasets like the weather classification dataset at hand.

Experiment 8 with C value 100 had the best accuracy at 94.24%, but the 0.0004% improvement was insignificant and threatened overfitting. Experiment 4 with a C value of 10 using a "lbfgs" solver was as good with an accuracy of 94.21% but was more computationally intensive without obvious advantages. Experiment 6 with a C value of 1 offered the best connection between regularization and efficiency. Experiment 6 is the strongest model, and it provides effective generalization together with computational feasibility, so this is the strongest model for the given dataset.

b) Decision Trees

Decision Trees are a well-known machine learning technique applied in classification and regression problems, where they have the tree-like nature that makes decision-making processes simple. They are known for being interpretable and capable of processing both numerical and categorical data easily. Decision Trees work by splitting the dataset into subsets recursively, depending on the most important features, to produce a model with the nature of a flowchart of decisions. This technique enables easy visualization of the decision-making process and is thus within reach even for non-specialists (Geeksforgeeks, 2025) [3].

The model of Decision Tree Classifier is made with the help of DecisionTreeClassifier class from sklearn.tree, which is a class that works best when machine learning is modeled on multi-class classification type problems with somewhat of complex patterns. The variable "Weather Type" was directly put as target variable without encoding, as decision trees can internally handle categorical data.

The dataset was standardized using StandardScaler to get all numeric features on some similar scale closer to one other so that there would be more probability that model would rightly identify splits deeper into model.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold, cross_val_score, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score  # Added metrics import

# Load dataset
file_path = "cleaned - weather_classification_data.csv"
df = pd.read_csv(file_path)

# Features and target
X = df.drop(columns=["Weather Type"])
y = df["Weather Type"]

# Standardize numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Fig. 31. DT code preparation and target variable initialization

The parameters used are stated below:

criterion = "entropy" – Use the information gain measure for deciding the splits on features. It is assistive for making informative splits

max_depth = 5 – Controls the depth of the tree to avoid overfitting and for interpretability

min_samples_split = 100 – ensures that at least 100 samples are available for a split; this lessens the overfitting

min_samples_leaf = 50 – Set at least 50 samples for a leaf node so that weak branches stay robust.

```
model = DecisionTreeClassifier(criterion = "entropy", random_state=42,
                               max_depth=5, min_samples_split=100, min_samples_leaf=50)
```

Fig. 32. DT model definition

The Model performance was evaluated using the K-Fold cross-validation technique. This was repeated 5 times and average accuracy performed across all folds was computed to give reliable estimation for model performance.

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_scaled, y, cv=kf, scoring="accuracy")
```

Fig. 33. K-fold cross validation function

The model then have been fit to the complete dataset and evaluated using model.predict() on the test set with performance metrics reported via accuracy_score, precision_score, recall_score, and f1_score so as to provide a broader perspective into how the model performed by looking at varying aspects of performance.

```
model.fit(X_scaled, y)
y_pred = model.predict(X_scaled)

accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred, average="weighted")
recall = recall_score(y, y_pred, average="weighted")
f1 = f1_score(y, y_pred, average="weighted")
```

Fig. 34. Performance metrics initialization

With the help of model.feature_importances_, the model could estimate feature importance, while a bar plot had been plotted wherein areas of the plot revealing many would be viewed as the highest influencers in decision-making.

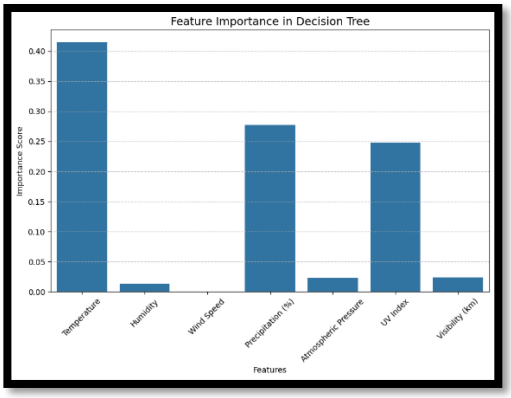


Fig. 35. Feature importance plot

The cross-validation accuracy plot was displayed in the form of a line chart determining how the performance was stable over the folds, providing an insight of the model's ability to generalize. A boxplot of accuracy distribution was also generated to assess variability across folds, for performance assessment.

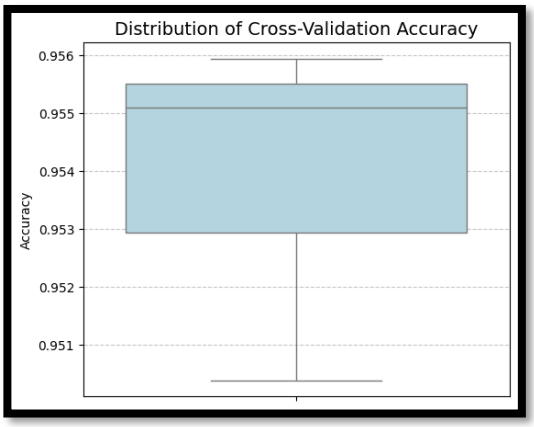


Fig. 36. Distribution of cross-validation accuracy plot

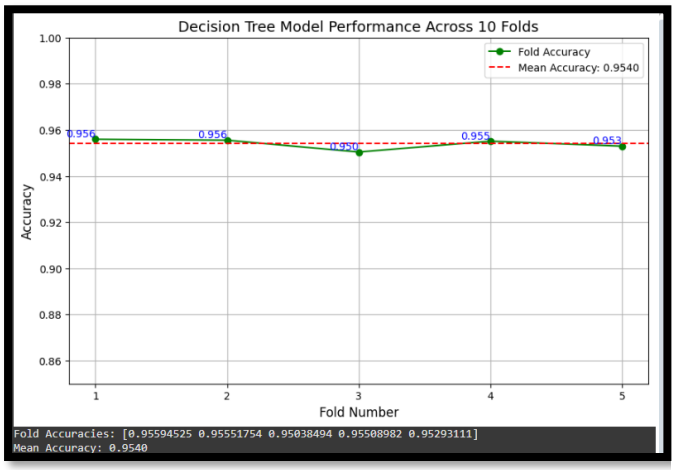


Fig. 37. Decision Tree Model Performance across folds

```
Classification Report - Decision Tree Model:
              precision    recall  f1-score   support

      Cloudy       0.96      0.92      0.94      2994
       Rainy       0.95      0.96      0.95      2828
       Snowy       0.98      0.99      0.98      2902
       Sunny       0.94      0.96      0.95      2965

    accuracy                           0.96     11689
   macro avg       0.96      0.96      0.96     11689
weighted avg       0.96      0.96      0.96     11689
```

Fig. 38. Performance Metrics Assessment

With the Decision Trees model prepared, the researcher then conducted experimentation through hyperparameter tuning using the following:

- max_depth
- min_samples_split
- min_samples,leaf
- criterion
- k-value (n_splits)

I.) Baselined DT Model

The baseline DT model contains the following parameters, with the visualization of the results being Fig. 34 to Fig. 37:

- max_depth = 5
- min_samples_split = 100
- min_samples,leaf = 50
- criterion = “entropy”
- n_splits = 5

The baseline decision tree model performed well, with a mean accuracy of 95.40% and very low variance between folds. The first and second fold delivered the highest accuracy level to 95.60%, while the third fold dropped to 95.04%, indicative of its consistency across each training subset. The model does exhibit very little variance in its accuracy, but that is also indicative of its stability and its ability to generalize quite well.

From feature importance analysis, Temperature was the main contributing variable, over .4, as it stands to be vital in weather classification. Precipitation and UV Index had also contributed to some significant degree, thus reinforcing their relevance in the prediction of weather. On the other hand, Humidity, Pressure, and Wind Speed had minor influence, favoring the suggestion they are not highly informative in this task.

The high model accuracy and balanced feature contribution clearly confirm the robustness and efficacy of the same model in handling a multi-class weather classification problem.

II.) Best Decision Tree Model

The best decision tree model contains the following parameters:

- max_depth = 10
- min_samples_split = 200
- min_samples,leaf = 100
- criterion = “gini”
- n_splits = 10

This decision tree model was highly accurate with a mean accuracy of 95.45% in 10-fold cross-validation, which signifies high consistency. The highest accuracy was 95.89% in the first and fourth folds, and the lowest recorded accuracy of 94.86% in the tenth fold still suggests consistent performance. The variance from one fold to another is small, indicating that the model is capable of generalizing well to new data.

Feature importance analysis identifies Temperature and Precipitation (%) as the most important variables, each with an importance score of more than 0.35. UV Index also had a strong impact, whereas Humidity, Wind Speed, and Pressure contributed little, indicating that environmental variables are most important in weather classification.

The report of the classification verifies overall balanced performance by weather categories, where precision, recall, and F1-score were averaging at 96%. High scores under Snowy and Rainy categories reflect that the model is properly able to identify similar patterns. Nevertheless, a small decline in Cloudy weather recall with the value of 92% signals scope for better performance in the same category.

The model was optimized on the gini criterion with the maximum depth at 10, minimum samples necessary to split at 200, and minimum samples per leaf set at 100. These hyperparameters helped in preventing overfitting while maximizing predictive accuracy. The tradeoff between depth and sample requirements assisted in the generalization and robustness of the model.

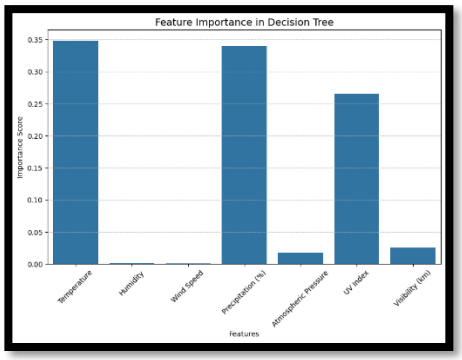


Fig. 39. Feature Importance of the best DT model

```
Classification Report - Decision Tree Model:
              precision    recall  f1-score   support

      Cloudy       0.96      0.92      0.94      2994
       Rainy       0.95      0.96      0.95      2828
       Snowy       0.98      0.99      0.98      2902
       Sunny       0.94      0.96      0.95      2965

    accuracy                           0.96     11689
   macro avg       0.96      0.96      0.96     11689
weighted avg       0.96      0.96      0.96     11689
```

Fig. 40. Performance Metrics of the best DT model

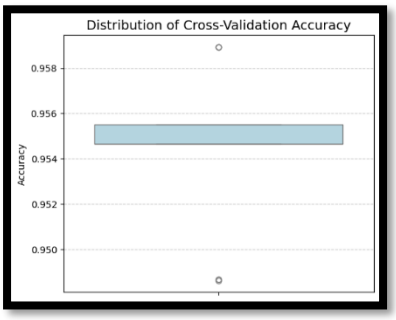


Fig. 41. Distribution of cross-validation accuracy of the best DT model

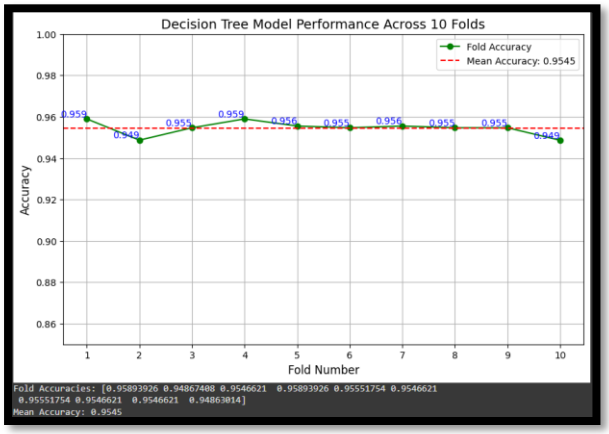


Fig. 42. Best DT model performance across folds and mean accuracy

III.) Worst Decision Tree Model

This decision tree model, experiment 5, contains the following parameters:

- max_depth = 7
- min_samples_split = 4900
- min_samples,leaf = 50
- criterion = “gini”
- n_splits = 10

The Decision Tree model generated an average accuracy of 70.86% over 8 folds, representing a significant drop from the previous model's 95.45% accuracy. The model's poor performance can be attributed to the higher regularization parameters, such as a large min_samples_split of 5000 and min_samples_leaf of 50, which made the model simpler and more biased. This decrease in complexity causes the model to lose precision

in picking out finer patterns in the data and results in underfitting. The fewer number of splits from 8 to 10 also increases the variance between folds, which leads to unreliable performance.

Although the reduced mean accuracy, the classification report reveals a good general accuracy of 93% with high precision and recall across most weather classes. Snow and Sunny weather were well predicted with good consistency, demonstrating that the model still maintains basic weather trends. Analysis of feature importance indicates that Temperature, Precipitation (%), and UV Index are the top predictors; Humidity and Wind Speed contribute negligible values. This means that the reduced model is biased towards keeping the most significant features at the expense of decreased overall accuracy but with increased generalization.

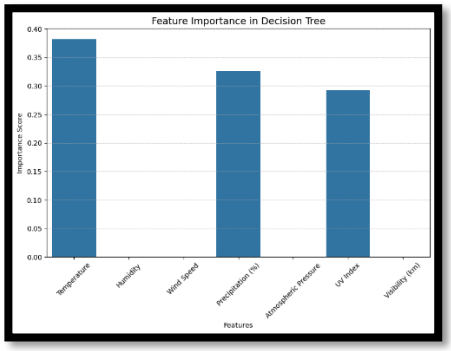


Fig. 43. Feature importance of the DT model 5

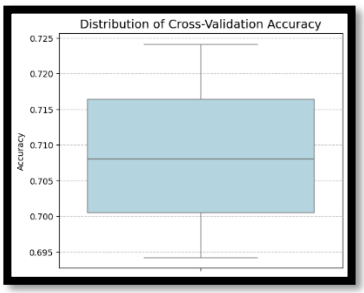


Fig. 44. Distribution of cross-validation accuracy of the DT model 5

```
Classification Report - Decision Tree Model:
              precision    recall  f1-score   support

      Cloudy       1.00      0.85      0.92      2994
       Rainy       0.91      0.94      0.92      2828
       Snowy       0.98      0.99      0.98      2902
       Sunny       0.85      0.95      0.90      2965

    accuracy                           0.93     11689
   macro avg       0.94      0.93      0.93     11689
weighted avg       0.94      0.93      0.93     11689
```

Fig. 45. Performance Metrics of the DT model 5

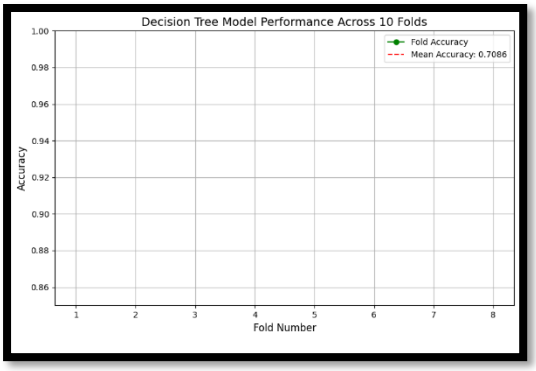


Fig. 46. DT model 5 performance across folds and mean accuracy

IV.) Decision Trees experiment assessment

The following figure then shows the complete experiment logs, utilizing conditional formatting to visualize the best and worst accuracies and containing each fold accuracy:

	Experiment No.	Algorithm	Hyperparameter 1	Hyperparameter 2	Hyperparameter 3	Hyperparameter 4	K Value	Overall Mean Accuracy
	1	Decision Trees	max_depth = 5	min_samples_split=100	min_samples_leaf=50	criterion = "entropy"	5	0.9540
	2	Decision Trees	max_depth = 8	min_samples_split=1000	min_samples_leaf=100	criterion = "entropy"	10	0.9358
	3	Decision Trees	max_depth = 7	min_samples_split=4900	min_samples_leaf=50	criterion = "gini"	10	0.8857
	4	Decision Trees	max_depth = 10	min_samples_split=2500	min_samples_leaf=200	criterion = "entropy"	8	0.9282
	5	Decision Trees	max_depth =10	min_samples_split=5000	min_samples_leaf=50	criterion = "gini"	8	0.7086
	6	Decision Trees	max_depth = 6	min_samples_split=100	min_samples_leaf=100	criterion = "entropy"	5	0.9492
	7	Decision Trees	max_depth = 9	min_samples_split=2500	min_samples_leaf=50	criterion = "entropy"	6	0.93
BEST MODEL	8	Decision Trees	max_depth = 10	min_samples_split=200	min_samples_leaf=100	criterion = "gini"	10	0.9545
	9	Decision Trees	max_depth = 7	min_samples_split=500	min_samples_leaf=100	criterion = "entropy"	10	0.945
	10	Decision Trees	max_depth = 10	min_samples_split=5000	min_samples_leaf=50	criterion = "gini"	5	0.7125

Fig. 47. Decision Tree Experiment Logs

Among the 10 experiments, after carefully analyzing accuracy and consistency, Experiment 8 stands out as the best choice. While Experiment 1 and Experiment 9 also showed strong performance, Experiment 8 delivered the highest accuracy with stable validation results. Although Experiment 1 performed well as a baseline, I prefer Experiment 8 for its balance between depth and generalization. Experiment 9 also demonstrated good accuracy, but Experiment 8's configuration of max_depth of 10, gini criterion, and k value of 10 ensured the most robust performance. At the very least, having multiple experiments provides useful insights and a strong foundation for future improvements in model tuning.

IV. Performance Evaluation and Comparison

a) Performance Metrics Comparison

The two models were evaluated using accuracy, precision, recall, and F1-score, to measure their performance on classification. The Decision Tree model gave the highest mean accuracy but had a lot of variances between the folds. On the other hand, the Multinomial Logistic Regression models provided a more consistent performance across the board and was stable in its scores on all metrics, although less accurate.

b) Performance Metrics Table

The following table contains the values of the best model for each algorithm, with each performance metric recorded throughout the whole experimentation process:

Metric	Logistic Regression (Model 6)	Decision Tree (Model 8)	Better Model
Mean Accuracy	94.20%	95.45%	Decision Tree
Highest Fold Accuracy	95.0%	95.90%	Decision Tree
Lowest Fold Accuracy	93.5%	94.9%	Decision Tree
Accuracy Range	1.5%	1%	Decision Tree
Precision	94%	96%	Decision Tree
Recall	94%	96%	Decision Tree
F1 - Score	94%	96%	Decision Tree

Table 1. Compiled Performance Metrics Table

The Decision Tree (DT) model performed better than the Multinomial Logistic Regression (MLR) model in terms of overall accuracy and classification. DT recorded a mean accuracy of 95.45% compared to 94.20% for MLR. Both models recorded comparable peak fold accuracies of 95.90% for DT and 95.0% for MLR. Decision Trees also had a higher stability of consistency with their accuracy ranging a narrower 1.0% compared to the 1.5% in MLR. Decision Trees had a superior performance in terms of precision, recall, and F1-score at 96% in all aspects, confirming its strong predictive ability.

c) Strengths and Weaknesses of the two algorithms

The Logistic Regression model exhibited very high consistency and reliability across all experiments, with low variance and a narrow accuracy range. Even with varying data conditions, its performance remained constant, thus providing reliable predictions for unseen data. Its slightly lower accuracy and recall indicate it may have difficulty in capturing more complex patterns, especially when the data is highly nonlinear.

The Decision Tree model performed better than Logistic Regression in terms of overall accuracy and recall, with the mean and best accuracy outperforming Logistic Regression by a considerable margin. It was able to correctly interpret the patterns in certain splits of the data, which was a positive sign of its capacity to model complex relationships. The model was, however, unstable with a high variance in the accuracy range of other experiments, registering the lowest accuracy of 70.86% in Experiment 5.

d) More effective algorithm

The Decision Tree was better suited to this dataset from its greater mean accuracy of 95.45% and higher performance in recall, precision, and F1-score. Being able to distinguish patterns at particular data splits reveals that it's better at retaining complex relationships compared to Logistic Regression. Its sensitivity to noisy classes or class imbalance is shown with its high variance. Logistic Regression, though less precise, was more consistent with a small range of accuracy and robust performance between folds, so it is more reliable to make consistent predictions on new data.

V. Conclusion and Reflection

a) Key findings

Major findings from the exercise indicate that the Decision Tree was superior to Logistic Regression, considering its mean accuracy achieved at 95.45% compared to Logistic Regression's 94.20%. It produced greater precision, recall, and F1-score, and better classifier. Although, the performance of the Decision Tree models was unstable with some experiments achieving an accuracy that is in the range of 70%, yet the experiments produced great and minimal accuracy ranges, with the best model at 1%. By contrast, the Logistic Regression models provided more stable performance, with an accuracy that was much less variable across the different splits of the data with a value of 1.5% from logistic regression. The strength behind Decision Tree was increased predictive power through modeling of a larger complexity picture, whereas Logistic Regression performed well in uniform handling of a wide array of data scenarios.

In addition, it was a poor choice of dropping the Cloud Cover, Season and Location columns entirely in the data preprocessing phase, as it could have been properly converted these data and use it to better refine the model. Fortunately, the accuracies of the models produced performed excellently.

b) Working with real-world data

When the models were applied to the data from the real world, the researcher was able learn about the complexity behind how various patterns of data interact with model configuration and its predictive accuracy. Cross-validation was highly effective in demonstrating the importance of evaluating performance across multiple splits and ensuring that models generalize well to unseen data. The tuning of hyperparameters like tree depth or regularization strength was also important concerning model behavior. Decision Trees did best on various data splits, which would indicate its ability of capturing non-linear relationships, but Logistic Regression represented itself as much more stable, which suggested its dominance in simpler decision boundaries.

c) Challenges

Among other tasks, the problems encountered were the management of data imbalance and noise, which affected Decision Tree performance through high variance. The Decision Tree model suffered from instabilities across certain splits due to its sensitivity to noisy or imbalanced data, while Logistic Regression, being dependent on linear relationships, was limited in its ability to identify more complex patterns. Future strategies could involve looking into ensemble techniques like Random Forest or Gradient Boosting to stabilize Decision Tree performance and enhance Logistic Regression's flexibility through tunings of regularization strength or feature scaling. An even better adeptness to the real-life complexities will come with balancing the data distribution and strong feature engineering in both models.

VI. References

[1] N. Nikhil, "Weather Type Classification," Kaggle, Jun. 2024. [Online]. Available: https://www.kaggle.com/datasets/nikhil7280/weather-type-classification. [Accessed: 14-Mar-2025].

[2] GeeksforGeeks, "Understanding Logistic Regression," GeeksforGeeks, Feb. 2025 [Online]. Available: https://www.geeksforgeeks.org/understanding-logistic-regression/. [Accessed: 14-Mar-2025].

[3] GeeksforGeeks, "Decision Tree," GeeksforGeeks, Jan. 2025. [Online]. Available: https://www.geeksforgeeks.org/decision-tree/. [Accessed: 14-Mar-2025].

[4] GeeksforGeeks, "Handling Missing Values," GeeksforGeeks, Aug. 2024 [Online]. Available: https://www.geeksforgeeks.org/ml-handling-missing-values/. [Accessed: 14-Mar-2025].

[5] Future Processing, "Data preprocessing: a comprehensive step-by-step guide," Future Processing, Aug. 2023. [Online]. Available: https://www.future-processing.com. [Accessed: 14-Mar-2025].

[6] GeeksforGeeks, "Training Data vs Testing Data," 2025 [Online]. Available: https://www.geeksforgeeks.org/training-data-vs-testing-data/. [Accessed: 14-Mar-2025].

[7] DataCamp, "K-Fold Cross Validation Explained," DataCamp, Jun. 2024 [Online]. Available: https://www.datacamp.com/tutorial/k-fold-cross-validation. [Accessed: 14-Mar-2025].

[8] Una'a-Hermosilla et. al., " Multinomial Logistic Regression to Estimate and Predict the Perceptions of Individuals and Companies in the Face of the COVID-19 Pandemic in the Ñuble Region, Chile," MDPI, 2022. [Online]. Available: https://www.mdpi.com/2071-1050/12/22/9553

[9] GeeksforGeeks, "Understanding the Confusion Matrix in Machine Learning", Feb. 2025 [Online]. Available: https://www.geeksforgeeks.org/confusion-matrix-machine-learning/ [Accessed: 14-Mar-2025].

[9] I. Belcic and C. Stryker, "What is hyperparameter tuning?" IBM, Jul. 23, 2024. [Online]. Available: https://www.ibm.com/think/topics/hyperparameter-tuning. [Accessed: 14-Mar-2025].

[10] M. Gupta, “Data Visualization in Machine Learning: Why it is Important?” Oct. 2024 [Online]. Available: https://www.appliedaicourse.com/blog/data-visualization-in-machine-learning/#:~:text=Data%20visualization%20plays%20a%20crucial,when%20looking%20at%20raw%20data. [Accessed: 14-Mar-2025].